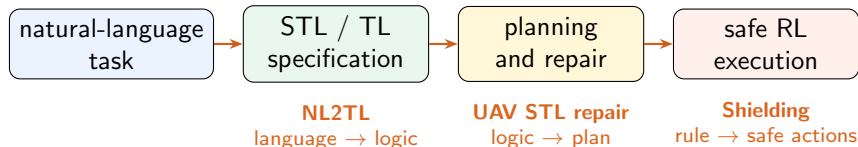


From Natural Language to Safe RL: Three Complementary Pieces



Main message

The papers form a natural pipeline, but each starts and stops at a different boundary. Together they motivate a focused gap: **natural-language safety requirements should become grounded STL constraints that are usable and measurable inside RL.**

Paper 1: NL2TL (EMNLP 2023)

Problem

Translate natural-language instructions into temporal logic across domains, despite limited manually labeled NL-TL pairs.

Method

- ▶ Replace concrete objects/actions with placeholders: `prop_1`, `prop_2`, ...
- ▶ Fine-tune T5 on lifted NL-STL pairs to learn temporal operators and formula structure.
- ▶ Use GPT-3 to synthesize data and detect atomic propositions in full natural language.

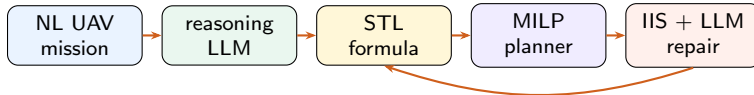
Example

“eventually visit A”
“and always avoid B”
↓
 $\mathbf{F prop_1} \wedge \mathbf{G} \neg \mathbf{prop_2}.$

Takeaway

T5-large reaches about 97.5% exact match on lifted data; full NL-to-STL with GPT-3 AP detection is around 95%–97%. The paper is a strong **language-to-specification front end**, not a safety execution method.

Paper 2: UAV NL-to-STL with MILP Repair (2026)



Core idea

The paper connects translation to execution: STL satisfaction and UAV dynamics are encoded into a mixed-integer linear program. If no trajectory exists, IIS diagnosis maps conflicts back to STL subformulas, and an LLM chooses a repair direction.

Safety-aware repair

$$\mathbf{G}_{[0,T]} \neg R_o \wedge \mathbf{F}_{[0,20]} R_g \rightsquigarrow \mathbf{G}_{[0,T]} \neg R_o \wedge \mathbf{F}_{[0,30]} R_g.$$

The obstacle-avoidance rule remains hard; only the deadline is relaxed.

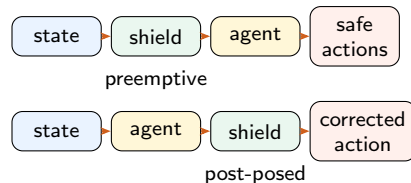
Paper 3: Safe RL via Shielding (2017)

Problem

RL may maximize reward while violating safety during training or deployment. Reward penalties alone do not provide formal guarantees.

Method

Given a temporal-logic safety rule φ_s and a conservative finite-state abstraction, synthesize a shield that filters or corrects unsafe actions before they can force a future violation.



Takeaway

Shielding is the strongest RL safety-execution piece, but it assumes the formal safety rule and environment abstraction are already available.

What Each Paper Gives, and What Is Still Missing

Paper	Solves	Strength	Missing for us
NL2TL	Natural language $\rightarrow$ STL/TL formula	Strong cross-domain translation via lifted APs and T5	No trajectory-level safety or RL execution
UAV STL repair	STL formula $\rightarrow$ feasible UAV trajectory or repaired task	Formal MILP feasibility and safety-preserving repair	Model-based planning, not policy learning
Shielding	Given safety rule $\rightarrow$ safe RL actions	Runtime enforcement during learning and execution	Assumes expert-written rule and abstraction

Synthesis

The missing piece is not another isolated translator, planner, or shield. It is the interface that turns **natural-language safety requirements** into **grounded STL constraints** whose effect can be evaluated on RL trajectories.

Our Focused Problem

Problem definition

Given an RL environment M and a language task $L = (L_{\text{goal}}, L_{\text{safe}})$, translate and ground L_{safe} into an STL safety constraint φ_{safe} , then train or evaluate policies under trajectory-level safety:

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^H \gamma^t r_t \right] \quad \text{s.t.} \quad \Pr_{\tau \sim \pi} [\tau \models \varphi_{\text{safe}}] \geq 1 - \delta.$$

First-stage scope

- ▶ Focus on the safety part of language tasks.
- ▶ Ground vague requirements into predicates and time windows.
- ▶ Evaluate violation rate, STL robustness, task success, and intervention count.

Why it follows naturally

- ▶ NL2TL gives translation, but not enforcement.
- ▶ UAV repair gives feasibility, but not RL learning.
- ▶ Shielding gives RL enforcement, but not language grounding.